

EC200U&EG91xU Series

QuecOpen(SDK) MMS API

Reference Manual

LTE Standard Module Series

Version: 1.1

Date: 2023-09-12

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-11-05	Herry GENG	Creation of the document
1.0	2021-11-24	Herry GENG	First official release
1.1	2023-09-12	Kruskal ZHU	1. Added applicable modules EG912U-GL and EG915U series. 2. Added a member QL_MMS_FS_FAIL to the MMS API result code enumeration mms_result_code_e (Chapter 3.4.1.1).

Contents

About the Document.....	3
Contents	4
Table Index.....	5
1 Introduction	6
1.1. Applicable Modules	6
2 MMS Interfaces Calling Process	7
3 MMS API.....	9
3.1. Header File	9
3.2. Demo	9
3.3. API Overview	9
3.4. API Description.....	10
3.4.1. ql_mms_client_new.....	10
3.4.1.1. mms_result_code_e	10
3.4.1.2. mms_send_result_cb	11
3.4.2. ql_mms_client_send_msg	12
3.4.2.1. mms_option_t.....	12
3.4.2.2. mms_charset_type	13
3.4.2.3. mms_send_param_t.....	14
3.4.2.4. mms_valid_type.....	14
3.4.2.5. mms_priority_type	15
3.4.2.6. mms_visible_type	16
3.4.2.7. mms_class_type.....	16
3.4.2.8. mms_info_t.....	17
3.4.2.9. mms_subject_t.....	17
3.4.2.10. mms_addr_t.....	18
3.4.2.11. mms_attachment_t.....	18
3.4.3. ql_mms_client_release	19
4 Appendix References	20

Table Index

Table 1: Applicable Modules 6

Table 2: API Overview 9

Table 3: Related Documents..... 20

Table 4: Terms and Abbreviations 20

1 Introduction

Quectel LTE Standard EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on SDK build environment. It introduces MMS interfaces calling process and API description of Quectel EC200U series and EG91xU family modules.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 MMS Interfaces Calling Process

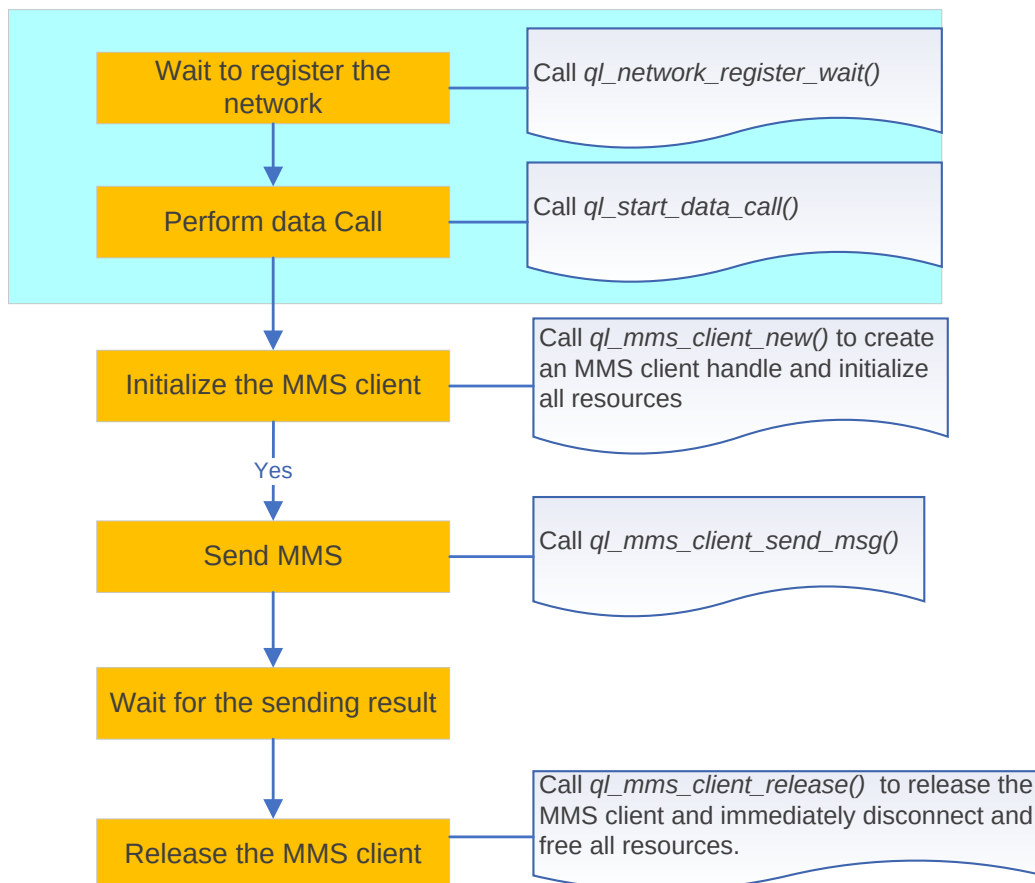


Figure 1: MMS Interface Calling Process

MMS sends and receives data based on the specified data channel, so you need to register the network and perform data call to establish the data channel (as shown in the figure above with blue background) before sending MMS. If the data channel has been established in other tasks, you can skip the steps of “Wait to register the network” and “Perform data call”, and call `ql_mms_client_new()` directly to create an MMS client handle. For more details about data call API, see **document [2]**.

After successful network registration and data call and the creation of an MMS client handle by calling `ql_mms_client_new()`, the MMS can be sent by calling `ql_mms_client_send_msg()`. The API returns immediately and calls the corresponding callback function when relevant events occur.

NOTE

1. EC200U series and EG91xU family QuecOpen modules only support MMS 1.3.
2. One simultaneous multimedia message request can only be fulfilled by the same MMS client. The next request can only be executed after the current request is completed. If you need to execute multiple requests for sending MMS at the same time, you need to create multiple MMS clients.

3 MMS API

3.1. Header File

ql_mms_client.h, the header file of MMS API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all the header files mentioned in this document are in this directory.

3.2. Demo

The demo codes file of MMS client is in the *\components\ql-application\mms\mms_demo.c* directory of the SDK.

3.3. API Overview

Table 2: API Overview

Function	Description
<i>ql_mms_client_new()</i>	Creates an MMS client handle and initializes MMS client resources.
<i>ql_mms_client_send_msg()</i>	Sends an MMS message.
<i>ql_mms_client_release()</i>	Releases the MMS client handle and relevant resources.

3.4. API Description

3.4.1. ql_mms_client_new

This function creates an MMS client handle and initializes MMS client resources.

● Prototype

```
int ql_mms_client_new(ql_mms_client *client_hdl, mms_send_result_cb result_cb, void *arg)
```

● Parameter

client_hdl:

[Out] MMS client handle.

result_cb:

[In] A callback function that informs the application of MMS sending results, see **Chapter 3.4.1.2** for details.

arg:

[In] Callback function of *result_cb*.

● Return Value

See **Chapter 3.4.1.1** for details.

3.4.1.1. mms_result_code_e

The enumeration of result codes:

```
typedef enum{
    QL_MMS_OK                      = 0,
    QL_MMS_INVLIAD_PARAM          = -1,
    QL_MMS_WOULDBLOCK             = -2,
    QL_MMS_OUT_OF_MEM             = -3,
    QL_MMS_HTTP_REQ_FAIL          = -4,
    QL_MMS_OPT_BUSY               = -5,
    QL_MMS_HTTP_RESP_FAIL         = -6,
    QL_MMS_SEND_FAIL              = -7,
    QL_MMS_HTTP_TIMEOUT           = -8,
    QL_MMS_DNS_FAIL               = -9,
    QL_MMS_FS_FAIL                = -10,
}mms_result_code_e;
```

● Member

Member	Description
<code>QL_MMS_OK</code>	Successful execution.
<code>QL_MMS_INVALID_PARAM</code>	Invalid parameter.
<code>QL_MMS_WOULDBLOCK</code>	Operation blocked.
<code>QL_MMS_OUT_OF_MEM</code>	Insufficient memory.
<code>QL_MMS_HTTP_REQ_FAIL</code>	Failed to send HTTP request.
<code>QL_MMS_OPT_BUSY</code>	The client is sending MMS.
<code>QL_MMS_HTTP_RESP_FAIL</code>	HTTP response failed.
<code>QL_MMS_SEND_FAIL</code>	Failed to send MMS.
<code>QL_MMS_HTTP_TIMEOUT</code>	HTTP request timeout.
<code>QL_MMS_DNS_FAIL</code>	Failed to parse domain name.
<code>QL_MMS_FS_FAIL</code>	MMS attachment <i>file_path</i> is invalid.

3.4.1.2. mms_send_result_cb

This function informs the application of MMS sending results.

● Prototype

```
typedef void(*mms_send_result_cb)(ql_mms_client client_hdl, int mms_result, int http_code, char *mms_err_msg, void *arg)
```

● Parameter

client_hdl:

[In] MMS client handle, obtained by `ql_mms_client_new()`.

mms_result:

[In] MMS sending result, see **Chapter 3.4.1.1** for details.

http_code:

[In] HTTP response code, see **document [3]** for details.

mms_err_msg:

[In] When *mms_result* is `QL_MMS_SEND_FAIL`, *mms_err_msg* gives the specific failure reason.

arg:

[In] Callback function, set by *ql_mms_client_new()*.

- **Return Value**

None

3.4.2. ql_mms_client_send_msg

This function sends an MMS message.

- **Prototype**

```
int ql_mms_client_send_msg(ql_mms_client *client_hdl, struct mms_option_t *send_option, struct mms_info_t *mms_info)
```

- **Parameter**

client_hdl:

[In] MMS client handle, obtained by *ql_mms_client_new()*.

send_option:

[In] Configuration for sending MMS, see **Chapter 3.4.2.1** for details.

mms_info:

[In] MMS content, including subject, recipient and attachment, see **Chapter 3.4.2.8** for details.

- **Return Value**

See **Chapter 3.4.1.1** for details.

3.4.2.1. mms_option_t

The structure of configuration for sending MMS:

```
struct mms_option_t{
    int                context_id;
    int                sim_id;
    char               *mmsc_url;
    char               *proxy_addr;
    uint16_t           proxy_port;
    int                charset;
    uint8_t            supportfield;
    int                wait_timeout;
    struct mms_send_param_t  send_param;
};
```

● Parameter

Type	Parameter	Description
int	<i>context_id</i>	Data channel number.
int	<i>sim_id</i>	SIM card ID.
char *	<i>mmmc_url</i>	MMSC address.
char *	<i>proxy_addr</i>	MMS agent address.
uint16_t	<i>proxy_port</i>	MMS agent port.
int	<i>charset</i>	MMS encoding modes, see Chapter 3.4.2.2 for details.
uint8_t	<i>supportfield</i>	Whether it is necessary to carry x-up-calling-id in the HTTP header.
int	<i>wait_timeout</i>	Maximum timeout for sending.
<i>struct mms_send_param_t</i>	<i>send_param</i>	Parameters to be used in sending MMS, see Chapter 3.4.2.3 for details.

3.4.2.2. mms_charset_type

The enumeration of MMS encoding modes:

```
typedef enum{
    MMS_CHARSET_ASCII_VALUE        = 0x03,
    MMS_CHARSET_UTF8_VALUE         = 0x6A,
    MMS_CHARSET_UCS2_VALUE         = 0x03E8,
    MMS_CHARSET_BIG5_VALUE         = 0x07EA,
    MMS_CHARSET_GBK_VALUE          = 0x71,
}mms_charset_type;
```

● Member

Member	Description
<i>MMS_CHARSET_ASCII_VALUE</i>	ASCII
<i>MMS_CHARSET_UTF8_VALUE</i>	UTF8
<i>MMS_CHARSET_UCS2_VALUE</i>	UCS2
<i>MMS_CHARSET_BIG5_VALUE</i>	BIG5
<i>MMS_CHARSET_GBK_VALUE</i>	GBK

3.4.2.3. mms_send_param_t

The structure of parameters to be used in sending MMS:

```
struct mms_send_param_t{
    int        validity_period;
    int        priority;
    int        delivery_report;
    int        read_report;
    int        visible;
    int        class;
};
```

● Parameter

Type	Parameter	Description
int	<i>validity_period</i>	MMS validity, see Chapter 3.4.2.4 for details.
int	<i>priority</i>	MMS priority, see Chapter 3.4.2.5 for details.
int	<i>delivery_report</i>	0 indicates that it is not necessary to submit a report. 1 indicates that it is necessary to submit a report.
int	<i>read_report</i>	0 indicates that it is not necessary to read a report. 1 indicates that it is necessary to read a report.
int	<i>visible</i>	MMS visibility, see Chapter 3.4.2.6 for details.
int	<i>class</i>	MMS type, see Chapter 3.4.2.7 for details.

3.4.2.4. mms_valid_type

The enumeration of MMS validity:

```
typedef enum {
    MMS_PARAM_VALID_1HOUR=0,
    MMS_PARAM_VALID_12HOURS,
    MMS_PARAM_VALID_24HOURS,
    MMS_PARAM_VALID_2DAYS,
    MMS_PARAM_VALID_1WEEK,
    MMS_PARAM_VALID_MAXIMUM,
    MMS_PARAM_VALID_DEFAULT
}mms_valid_type;
```

- Member

Member	Description
<i>MMS_PARAM_VALID_1HOUR</i>	Validity period: 1 hour.
<i>MMS_PARAM_VALID_12HOURS</i>	Validity period: 12 hours.
<i>MMS_PARAM_VALID_24HOURS</i>	Validity period: 24 hours.
<i>MMS_PARAM_VALID_2DAYS</i>	Validity period: 2 days.
<i>MMS_PARAM_VALID_1WEEK</i>	Validity period: 1 week.
<i>MMS_PARAM_VALID_MAXIMUM</i>	Validity period: 0x7FFFFFFF seconds.
<i>MMS_PARAM_VALID_DEFAULT</i>	The validity period is not set but determined by the network.

3.4.2.5. mms_priority_type

The enumeration of MMS priority:

```
typedef enum {
    MMS_PARAM_PRIORITY_LOWEST=0,
    MMS_PARAM_PRIORITY_NORMAL,
    MMS_PARAM_PRIORITY_HIGHEST,
    MMS_PARAM_PRIORITY_DEFAULT
}mms_priority_type;
```

- Member

Member	Description
<i>MMS_PARAM_PRIORITY_LOWEST</i>	Lowest priority.
<i>MMS_PARAM_PRIORITY_NORMAL</i>	Normal priority.
<i>MMS_PARAM_PRIORITY_HIGHEST</i>	Highest priority.
<i>MMS_PARAM_PRIORITY_DEFAULT</i>	Default priority, determined by the network.

3.4.2.6. mms_visible_type

The enumeration of MMS visibility:

```
typedef enum {
    MMS_PARAM_SENDERADDR_HIDE=0,
    MMS_PARAM_SENDERADDR_SHOW,
    MMS_PARAM_SENDERADDR_DEFAULT
}mms_visible_type;
```

● Member

Member	Description
<i>MMS_PARAM_SENDERADDR_HIDE</i>	Hidden.
<i>MMS_PARAM_SENDERADDR_SHOW</i>	Visible.
<i>MMS_PARAM_SENDERADDR_DEFAULT</i>	Default, determined by the network.

3.4.2.7. mms_class_type

The enumeration of MMS type is defined as below:

```
typedef enum{
    MMS_PARAM_CLASS_PERSONAL=0,
    MMS_PARAM_CLASS_ADVERTISEMENT,
    MMS_PARAM_CLASS_INFORMATIONAL,
    MMS_PARAM_CLASS_AUTO,
    MMS_PARAM_CLASS_DEFAULT
}mms_class_type;
```

● Member

Member	Description
<i>MMS_PARAM_CLASS_PERSONAL</i>	Personal.
<i>MMS_PARAM_CLASS_ADVERTISEMENT</i>	Advertisement.
<i>MMS_PARAM_CLASS_INFORMATIONAL</i>	Informational.
<i>MMS_PARAM_CLASS_AUTO</i>	Automatic.
<i>MMS_PARAM_CLASS_DEFAULT</i>	Default, determined by the network.

3.4.2.8. mms_info_t

The structure of MMS content:

```
struct mms_info_t{
    struct mms_subject_t    *subject;
    struct mms_addr_t      *to_addr;
    struct mms_addr_t      *cc_addr;
    struct mms_addr_t      *bcc_addr;
    struct mms_attachment_t *attachments;
};
```

● Parameter

Type	Parameter	Description
<i>struct mms_subject_t *</i>	<i>subject</i>	Subject, see Chapter 3.4.2.9 for details.
<i>struct mms_addr_t *</i>	<i>to_addr</i>	Recipient address, see Chapter 3.4.2.10 for details.
<i>struct mms_addr_t *</i>	<i>cc_addr</i>	Carbon copy recipients address, see Chapter 3.4.2.10 for details.
<i>struct mms_addr_t *</i>	<i>bcc_addr</i>	Blind carbon copy recipients address, see Chapter 3.4.2.10 for details.
<i>struct mms_attachment_t *</i>	<i>attachments</i>	Attachment information, see Chapter 3.4.2.11 for details.

3.4.2.9. mms_subject_t

The structure of MMS subject:

```
struct mms_subject_t{
    char    *subject;
    int     length;
};
```

● Parameter

Type	Parameter	Description
char *	<i>subject</i>	Subject content.
int	<i>length</i>	Length of subject content.

3.4.2.10.mms_addr_t

The structure of addresses of recipients, cc and bcc recipients of MMS:

```
struct mms_addr_t{
    struct mms_addr_t *next;
    int                addr_len;
    char               *address;
};
```

● Parameter

Type	Parameter	Description
<i>struct mms_addr_t *</i>	<i>next</i>	Next node.
int	<i>addr_len</i>	Length of number or address.
char *	<i>address</i>	Number or address.

3.4.2.11.mms_attachment_t

The structure of MMS attachment:

```
struct mms_attachment_t{
    struct mms_attachment_t *next;
    char                    *file_path;
};
```

● Parameter

Type	Parameter	Description
<i>struct mms_attachment_t *</i>	<i>next</i>	Next node.
char *	<i>file_path</i>	File path where the attachment is located.

3.4.3. ql_mms_client_release

This function releases the MMS client handle and relevant resources.

- **Prototype**

```
int ql_mms_client_release(ql_mms_client *client_hdl)
```

- **Parameter**

client_hdl:

[In] MMS client handle, obtained by *ql_mms_client_new()*.

- **Return Value**

See **Chapter 3.4.1.1** for details.

4 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Data_Call_API_Reference_Manual
[3] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_HTTP_API_Reference_Manual

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
DNS	Domain Name Server
GBK	Chinese Internal Code Extension Specification
HTTP	Hypertext Transfer Protocol
ID	Identifier
IoT	Internet of Things
LTE	Long-Term Evolution
MMS	Multimedia Messaging Service
MMSC	Multimedia Messaging Service Center
RTOS	Real-Time Operating System
SDK	Software Development Kit

SIM	Subscriber Identity Module
UCS	Universal Character Set
USB	Uniform Resource Locator
UTF	Unicode Transformation Format